Title & Mapping

Paper Title: ScaleMCP: Dynamic and Auto-Synchronizing Model Context Protocol Tools for LLM Agents
Authors & Year: Elias Lumer, Anmol Gulati, Vamse Kumar Subbiah, Pradeep Honaganahalli Basavaraju, James A. Burke - 2025
Student Details: Roma Pai, 2024AA05965
Paper Link: https://arxiv.org/abs/2505.06416
I**nstructor-Provided Topic Mapping**: L1.3: The Protocol Landscape (MCP)

Problem & Motivation

The paper discusses scalability and maintenance issues in connecting LLMs to external tools:

- The "Context Window" Constraint: LLMs and providers (like OpenAI) set strict limits on tool usage (eg not more than 128 tools at a time).
- The "Stale Data" Problem: Existing frameworks work on local tool repositories that need manual updates leading to duplication, human error, and inconsistencies between the tool definition and the retrieval system.
- Lack of Autonomy: Current methods simplify tool selection before the LLM is called. This prevents the agent from dynamically re-querying or finding new tools during a multi-turn conversation if the initial retrieval fails.
- Inefficient Retrieval Embeddings: Curent systems uses simple concat or average of tool documentation (names, descriptions, parameters) for vector storage. This approach does not focus on important elements like synthetic questions or specific tool names.

Core Idea & Key Components

The main idea of ScaleMCP is that tool selection becomes a dynamic, agent-controlled process instead of a static, pre-inference step. The integration of the MCP as the live "single source of truth," creates a self-healing pipeline that maintains the agent's retrieval memory in perfect sync with external server capabilities without human intervention.

Key Components

A. Auto-Synchronization Indexing Pipeline (Updater) : The system runs a continuous CRUD (Create, Read, Update, Delete) loop between MCP Servers and Vector/Graph Storage. This prevents "stale data" and avoids manual repository upkeep. It ensures the retriever does not deliver hallucinations based on old API schemas.

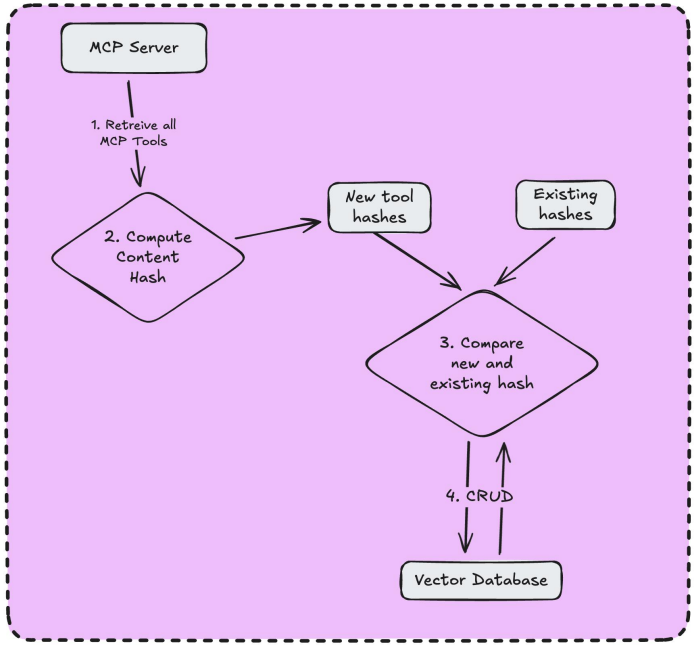
B. Tool Document Weighted Average (Encoder) - This embedding strategy replaces standard concat with a normalized weighted vector sum. It handles poor retrieval performance caused by poor embeddings, where critical keywords are lost in long API parameter lists.

C. The MCP Retrieval Tool (Router & Agentic Interface) - Instead of pushing tools to the agent, the system provides the LLM with a specific MCP_Retrieval_Tool allowing the agent to act independently. It can re-query if the initial results are not appropriate.

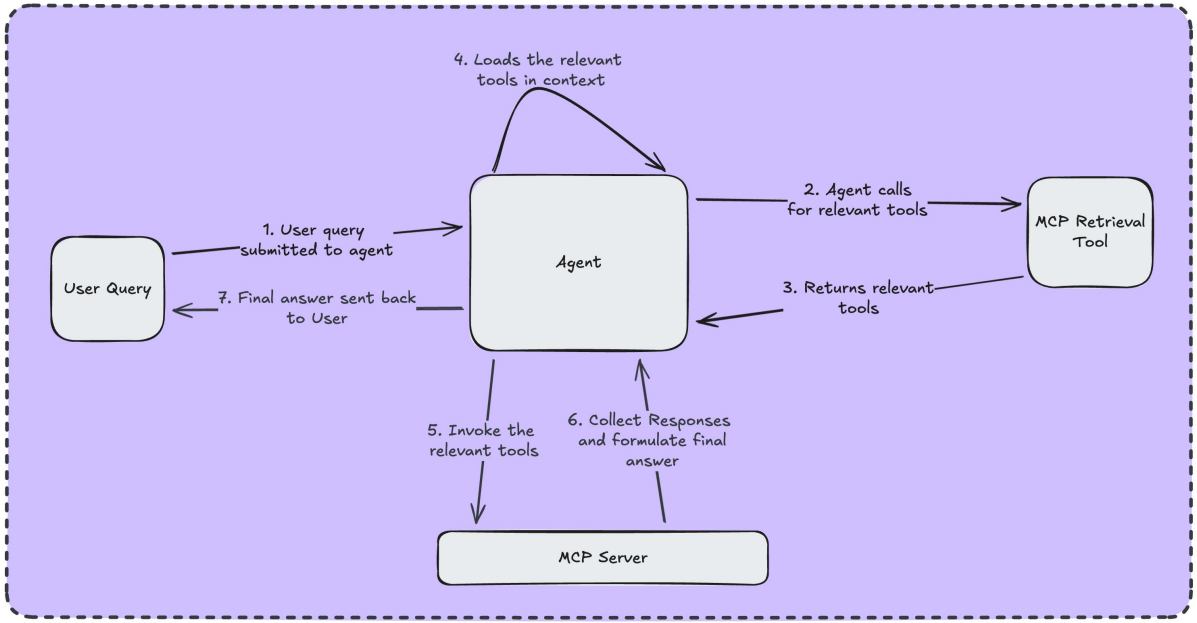
D. Scoring & Reranking Module (Refiner) - This is a stage after retrieval that filters out irrelevant tools that are semantically similar but functionally incorrect. It uses Cross-Encoder Reranking (eg: Cohere V3) or LLM-based Reranking (GPT 4o) on the initial vector search results.

Architecture Diagram

Phase1: Auto Synchronisation Indexing Pipeline (Background)



Phase2: LLM Agent Invocation (Realtime Interaction)



Architecture Explanation (Technical)

Phase 1: Auto-Synchronization Indexing Pipeline (Background Process)

- Single Source of Truth (MCP Servers): The architecture treats the MCP servers as the definitive source for tool definitions, ensuring consistency across the system.
- Content Hashing Algorithm: The system retrieves all available MCP tools and computes a SHA-256 hash for each by concatenating the tool's name, description, and argument parameters.
- State Comparison & Logic: These new hashes are compared against existing hashes stored in the index.
  - Match: If hashes match, no action is taken.
  - Mismatch: If a hash is new or changed, the system triggers specific CRUD (Create, Read, Update, Delete) operations.

• Vector Database & TDWA Embedding: When adding new tools to the Vector Database, the system utilizes the Tool Document Weighted Average (TDWA) strategy

Phase 2: LLM Agent Invocation (Real-Time Interaction)

1. The user submits a natural language query to the Agent.
2. The Agent autonomously invokes a specialized "MCP Retrieval Tool" using keywords derived from the user query. This component queries the synchronized Vector Database and returns the relevant tool definitions.
3. The system "binds" the retrieved MCP server definitions into the LLM's active context via function calling protocols, effectively expanding the agent's capabilities mid-interaction.
4. Once the tools are loaded, the Agent decides which specific MCP servers to call.
5. The Agent collects the raw outputs from the MCP servers, reasons through the data, and synthesizes a final natural language answer for the user.

Experimental Results / Observations

- The "Hallucinated Success" Paradox: GPT-3 achieved the highest task completion at 94.4% despite a low tool correctness of 36.1%. This shows the model often found the right answer using its own knowledge or incomplete context without properly making the correct tool call.
- Model Size does not equal Tool capability: Smaller models like GPT-4o-mini outperformed larger models, such as GPT-4.1, in tool correctness. This makes them cost-effective choices for ScaleMCP implementations.
- Simple text concatenation worked well for raw vector search. However, the TDWA (Weighted Average) embedding strategy showed advantages when combined with advanced rerankers, especially in improving the ordering of results based on MAP scores.

Correct Answers do not equal Correct Process - An agent can satisfy a user (High Task Completion) while failing to use the provided system tools correctly (Low Tool Correctness)

LLM Agent	Tool Correctness (%)	Task Completion (%)	Performance Profile
GPT-4o-mini	54.0%	86.7%	Best Balance (High precision & completion)
GPT-4o	51.7%	83.9%	Strong All-Rounder
GPT-o3	36.1%	94.4%	Highest Completion, Low Tool Accuracy
Claude 3.7	23.1%	69.4%	Underperformed in this specific setup

Your Analysis & Insights (Part 1)

1. Dynamic Link Libraries for LLMs

The core innovation of ScaleMCP is not just retrieval; it is the architectural shift from static pre-loading to dynamic binding.

- Traditional agents behave like "fat clients," carrying every possible tool in their context window (system prompt) at initialization. ScaleMCP effectively implements a Just-in-Time (JIT) compilation logic. The agent is initialized as a "thin client" with only one capability: the MCP_Retrieval_Tool.
- By creating a "tool that retrieves tools," the paper solves the context window paradox: To solve a complex task, you need complex tool schemas, but complex schemas consume the context needed for reasoning. ScaleMCP offloads the storage of capabilities to the vector database and only streams the execution logic when the intent is detected.

2. Assumptions

- The framework assumes tools are independent. If Tool B requires an ID returned by Tool A, the MCP_Retrieval_Tool must be smart enough to retrieve both based on the initial query, or the Agent must perform iterative retrieval. The paper's high performance on "parallel" calls implies the test queries were largely parallelizable, masking potential failures in sequential dependency chains.
- The "Auto-Sync" relies on the MCP server being the "Single Source of Truth". This assumes developers maintain perfect, descriptive docstrings in their code. If a developer updates code but leaves a vague description, the Hash changes (triggering an update) but the Embedding quality degrades, effectively poisoning the retrieval index automatically.

Your Analysis & Insights (Part 2)

3. Real-World Deployment Behavior & Friction

- The experimental results for GPT-o3 (94.4% Completion vs. 36.1% Tool Correctness) reveal a dangerous deployment reality. An agent might "guess" the right number (high completion) without actually querying the database (low correctness) resulting in a critical failure mode. ScaleMCP deployment requires strict Force-Calling protocols to prevent models from bypassing the retrieved tools.
- Even though the authors strongly suggest autonomy, the architecture introduces latency. A single user query now triggers:
    1. Agent Reasoning (Call Retrieval Tool)
    2. Vector Search + Cross-Encoder Reranking (Computationally expensive)
    3. Tool Binding (Context Update)
    4. Agent Reasoning (Call Actual Tool)
    5. Tool Execution
    - In real-time voice or chat applications, this multi-hop "stop-and-think" loop may degrade user experience compared to static tool definitions, creating a trade-off between scalability (infinite tools) and responsiveness.

Peer Feedback (Two Students)

Peer 1:
KUPPA GOWRI SANKAR (2024AA05936)
Positives: Good conceptual insights, clear structure

Negatives: Architecture technical slide is too much

Peer 2:
MANJUNATHA K N (2024AB05252)
Positives: clear problem understanding

Negatives: limited tradeoff discussion

Changes done: Simplify explanations, added examples, read more on when not to use ScaleMCP

LLM Usage Disclosure (Compulsory)

innovate achieve lead

1. AI Assistance Statement - Yes, an LLM was used

2. Scope of Usage

The AI was utilized for the following specific tasks:

- Paper Summarization
- Technical Simplification: To help clarify and format the mathematical notation for the TDWA

1. Extent of Assistance

The LLM served as a "reading assistant" and "editor." It helped me to summarise what the paper meant, and after writing down my intial thoughts it helped me to structure the presentation well, along with the presenter notes and the spelling mistakes.

4. I hereby affirm that:

- This presentation represent my own conceptual understanding of the material.
- All Diagrams and Visualizations presented were conceptualized or redrawn by me.